Connected Vehicle Intelligence Hackathon

Solution Document Template

Submission Format: PDF (export of this document)

To be Submitted by: Yashwanth1887 (individual submission)

Team Members & Roles: Yashwanth1887 ? Project owner / lead engineer ? email not provided for document

Problem Space Chosen: Connected fleet operations ? idle fuel waste, real-time telemetry intelligence and cost optimisation

Repository URL: https://github.com/Yashwanth1887/Real-Time-Fleet-Intelligence-Cost-Optimisation-Platform

Demo Video URL (? 5 min): Not recorded yet ? add the unlisted video link before portal submission

Date of Submission: 01/10/2026

Table of Contents (section order follows this template; use Word navigation or PDF bookmarks)

1. Executive Summary

2. Problem Statement & Validation

2.1 Problem Statement

2.2 Evidence & Validation

2.3 Impact & Success Metrics

3. Solution Description

3.1 Solution Overview & User Journey

3.2 Key Value Proposition

3.3 Innovative Ideas

4. Feature List

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

5.2 Technology Stack & Justification

5.3 Data Architecture

5.4 Deployment View

6. Low-Level Design

6.1 Layering & Separation of Concerns

6.2 Design Principles Applied

6.3 Design Patterns Used

6.4 Interfaces, Contracts & Runtime Flows

6.5 Algorithms & Data Structures

7. Non-Functional Requirements & Performance Benchmarks

8. Security & Compliance

9. Test Strategy

10. Observability

11. AI / ML Component (if used)

12. Architecture Decisions, Risks & Future Enhancements

13. Demo Video (5 Minutes Maximum)

14. Repository Checklist

15. Conclusion

16. Declarations

17. Appendix (if any)

1. Executive Summary

Brief overview (half a page maximum) of:

- The problem you chose and who it affects
- What your solution does, in one or two sentences
- Key results achieved (throughput, latency, impact numbers)
- Unique features, innovations or optimisations you implemented

FleetGuard is a local connected-vehicle intelligence prototype for fleet managers who need to connect vehicle behaviour to operational cost and a practical action. Synthetic telemetry flows through Kafka; a consumer validates events, stores raw and hot state, detects idle episodes, estimates cost and creates alerts and recommendations shown in an authenticated React dashboard. The local vertical slice and browser flow were verified. Producer smoke runs delivered 2,000 events at each of 100, 1,000 and 10,000 events/sec targets, observing 100.0, 998.6 and 9,218.3 generated/delivered events/sec respectively; lag was later observed at zero across three partitions. These are bounded local producer measurements, not full-pipeline capacity or a 100K events/sec result. No customer impact or realised savings are claimed.

2. Problem Statement & Validation

2.1 Problem Statement

- **Format:** “[User] needs a way to [job to be done] because [pain], which today costs [measurable impact].”
- **Primary User:** Fleet manager, dealer, lender, driver or OEM — name one
- **Secondary Stakeholders:** Others affected by the problem

Primary user: fleet manager. Need: ?A fleet manager needs a way to identify costly engine-on idling and act on it because scattered telemetry does not directly explain the operational cost or next step.? Quantified fleet-wide loss is not established in this project. Secondary stakeholders include operations analysts, drivers, OEM data providers and fleet owners. Facts: an implemented synthetic-data pipeline and deterministic idle-cost workflow are documented in this repository. Assumptions: idling is actionable and the manager benefits from cost-ranked, explainable recommendations. Validation method: local simulation, API/browser smoke checks and bounded producer load steps; no interviews, public fleet dataset or field pilot were conducted. Existing options include OEM portals, aftermarket telematics and fleet platforms; no comparative product study was performed, so differentiation is a prototype hypothesis, not a verified market gap.

2.2 Evidence & Validation

- **Facts vs Assumptions:** Separate what you know from what you assume
- **Validation Method:** Published study, public dataset, simulation, or interviews
- **Existing Alternatives:** Aftermarket dongles, OEM portals, Motorq Fuse, other telematics tools — and why they fall short

Evidence / Assumption	Source or Method	What It Shows	Confidence
Idling can contribute to avoidable fuel use in fleets	Product hypothesis; synthetic scenario only	Justifies testing an idle-cost workflow; no fleet loss estimate	Low
Event-to-action data path can be demonstrated locally	Compose smoke run and authenticated browser walkthrough	Validates prototype workflow, not customer value	Medium

2.3 Impact & Success Metrics

Metric	Baseline Today	Target	How Measured / Estimated
Idle-cost visibility	No measured baseline	Demonstrate linked alert/cost/action	Synthetic workflow inspection
Scale / throughput	No production baseline	100K events/sec is a case-study target	Local producer smoke only; 9,218.3/sec observed at 10K target

- **Scale of Impact:** Who benefits, by how much, at 10K vs 100K vehicles

- **Wider Impact:** Safety, cost, environment or compliance, where relevant

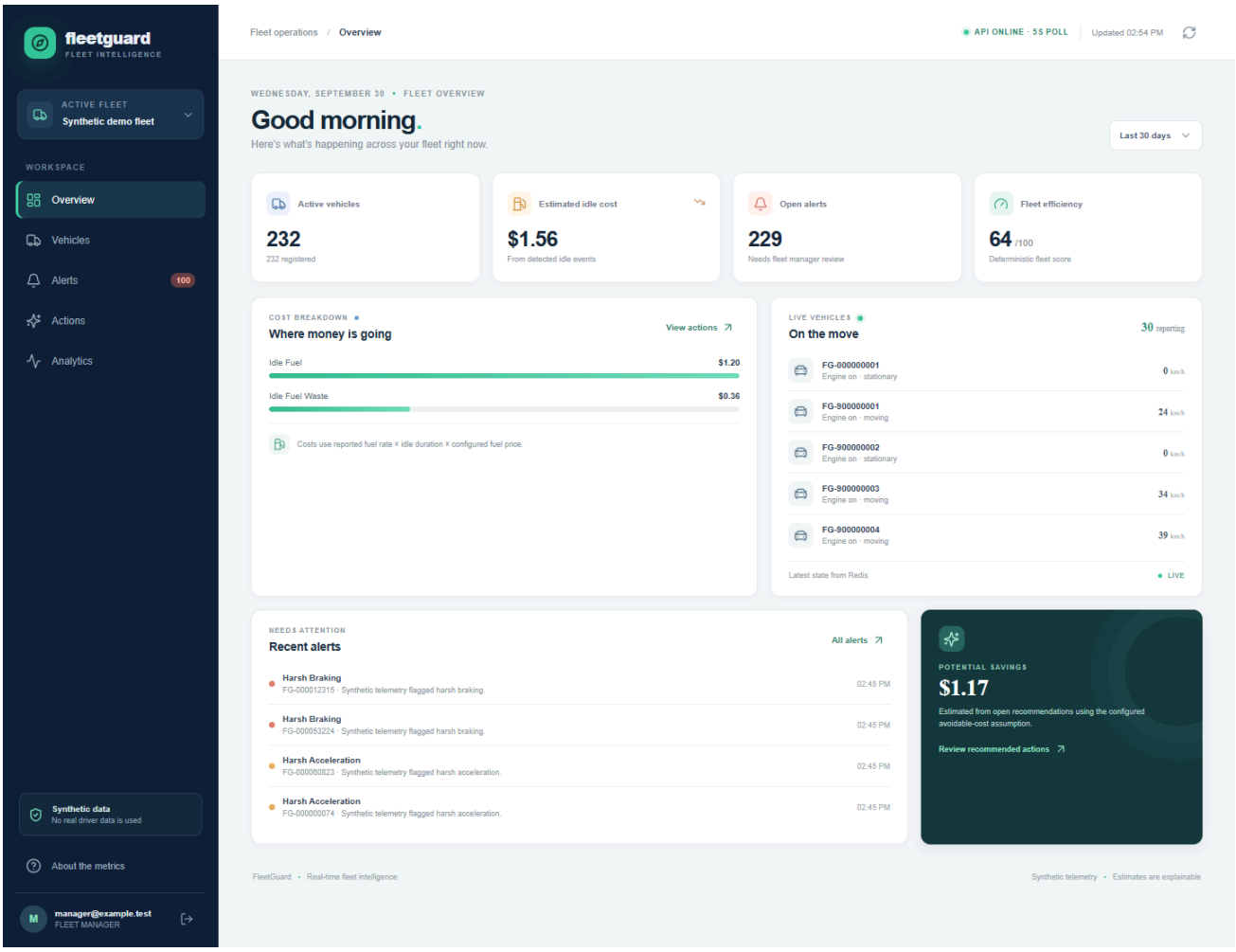
Impact is expressed as a scale model, not observed savings. The application?s simulator accepts an identifier pool of 100,000, but the test did not provision 100K active vehicles. For a planning model, event volume scales linearly with event rate: at 10K events/sec and 508 bytes/event average, raw JSON is approximately 408.8 GiB/day; at 100K events/sec, approximately 4,087.6 GiB/day. This excludes protocol overhead, indexes, replication, compression and retention. The system did not test those rates end-to-end. Potential benefit is earlier visibility into avoidable idle cost; no currency savings, safety improvement or emissions reduction was measured.

3. Solution Description

3.1 Solution Overview & User Journey

- **What It Does:** Plain-language description of the solution
- **User Journey:** Step by step, e.g. vehicle event → detection → alert → action → outcome
- **Screenshot:** At least one screenshot of the working product

A synthetic vehicle event is published to Kafka, validated by the consumer, stored in MongoDB, reflected in Redis current state and evaluated by deterministic rules. A qualifying idle episode writes an alert, cost event and recommendation to PostgreSQL; the manager signs in and reviews the linked explanation in the dashboard. The screenshot below shows the working dashboard from a local browser check. Demo records and locations are synthetic.



3.2 Key Value Proposition

- **Customer Job:** What the user is trying to get done
- **Pain Relieved:** What gets easier, cheaper or safer
- **Gain Created:** New capability the user did not have
- **Differentiation:** Why your solution, not the alternatives

Customer job: find the fleet issue that merits attention and understand what to do. The prototype brings telemetry, estimated cost and a linked recommendation into one workflow. It provides an explainable deterministic rule and local real-time event path. The value is a demonstrated capability, not measured customer savings. No head-to-head product comparison was conducted.

3.3 Innovative Ideas

- **Up to Three Ideas:** A technique, data insight, user experience or business model
- **What Is New:** Compared against prior work you studied
- **Evidence:** Proof that each idea works

1) ?WHY ? COST ? ACTION? links an idle signal to an explicit cost estimate and recommendation; evidence: synthetic end-to-end smoke event persisted the alert, cost and action. 2) A polyglot path separates durable relational action records, raw event history and expiring hot state; evidence: local Compose integration checks. 3) A simulator supports repeatable rates and edge scenarios; evidence: producer smoke measurements. These are implementation choices; novelty against prior art is not established.

4. Feature List

Every feature must be traceable to code. Priority uses MoSCoW (Must / Should / Could / Won't). Status: Done / Partial / Planned. Give the demo video timestamp for Done features.

ID	Feature	User Story	Priority	Status	Code Path	Video
F-01	Synthetic telemetry ingestion	As an operator, I want events ingested for timely fleet state	Must	Done	simulator/main.py; backend/app/workers/telemetry_consumer.py	No video; record after filming
F-02	Idle cost alert and recommendation	As a manager, I want estimated idle cost linked to an action	Must	Done	backend/app/services/idle_cost.py; backend/app/api/dashboard.py	No video; record after filming
F-03	Authenticated fleet dashboard	As a manager, I want to inspect vehicles, alerts and analytics	Should	Done	frontend/src/App.tsx; backend/app/api/	No video; record after filming

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

- **System Context Diagram:** C4 level 1 — your system, users and external systems (OEM clouds, identity provider, LLM API, data warehouse)
- **Container Diagram:** C4 level 2 — every service, data store and broker, with the protocol on each arrow (MQTT, HTTPS, gRPC, Kafka)
- **Data Flow:** Path of one telemetry event from vehicle to insight, with expected latency per hop

System context: Fleet manager ? FleetGuard dashboard/API; synthetic simulator ? Kafka; FleetGuard consumer ? MongoDB raw events, Redis current state and PostgreSQL alert/cost/action records. No OEM cloud, external identity provider or LLM is integrated. Container/data-flow and deployment descriptions are in docs/architecture/ and infra/docker-compose.yml. Per-hop latency was not instrumented; the UI polls on a configured five-second interval, which is not an end-to-end latency measurement. Local protocols: simulator? Kafka (Kafka protocol), consumer?stores (native drivers), browser?API (HTTPS in production; HTTP on local demo), API?PostgreSQL (SQL/TCP).

5.2 Technology Stack & Justification

Layer	Choice	Why This, and What You Rejected
Ingestion / Messaging	Apache Kafka	Replay and partition ordering; single local broker limits resilience
Stream / Batch Processing	Python consumer; analytics batch	Simple Python rules and batch job fit prototype; no stream framework
Relational / NoSQL / Cache / Search / Vector	PostgreSQL + MongoDB + Redis; no search/vector store	Relational action records, raw event documents and expiring current state by access pattern
Backend / Frontend	FastAPI, SQLAlchemy, React, TypeScript, Vite	Typed API and browser dashboard; no external SaaS dependency
ML / AI	None at runtime	Deterministic explainable rules; no training data/model
Infrastructure / CI-CD / Observability	Docker Compose; docs/logging/health checks	Portable local demo; no CI pipeline or metrics dashboard configured

5.3 Data Architecture

- **ER Diagram:** Relational core normalised to 3NF; list any deliberate denormalisation and why
- **Polyglot Map:** Which data lives in which store, with its CAP choice (CP or AP)
- **Capacity Estimate:** Events/sec, bytes/event, daily and yearly volume, partition keys, hot / warm / cold retention

- **Query Optimisation:** Top three queries with EXPLAIN ANALYZE before and after

Query	Before (ms)	After (ms)	Change Made
Top dashboard/API query baseline	Not measured	Not measured	No EXPLAIN ANALYZE capture; no optimization claim
Alert/cost join baseline	Not measured	Not measured	No EXPLAIN ANALYZE capture; no optimization claim
Vehicle list query baseline	Not measured	Not measured	No EXPLAIN ANALYZE capture; no optimization claim

5.4 Deployment View

- **Deployment Diagram:** Clusters, namespaces, replicas, autoscaling, networking, secrets
- **Cloud-Agnostic Approach:** How the same deployment runs on a second cloud or locally

The delivered deployment is Docker Compose on one local host: frontend, FastAPI backend, telemetry worker, Kafka, PostgreSQL, MongoDB and Redis. It uses one broker, one worker, three Kafka partitions and no autoscaling, namespaces or multi-zone redundancy. Secrets are supplied through ignored local .env files with committed .env.example templates. Network boundaries are Compose networks. The same containers can run on another Docker-capable host or cloud VM after supplying environment configuration and persistent volumes; managed services and cloud failover were not tested.

6. Low-Level Design

6.1 Layering & Separation of Concerns

- **Architecture Style:** Layered, hexagonal (ports and adapters) or clean architecture — justify
- **Layer Diagram:** Internal layers of at least one core service and the allowed dependency direction
- **Folder Structure:** Repository tree, two levels deep

Layer	Responsibility	Must Not
Presentation / API	HTTP routes, validation, auth checks, response schemas	Contain business rules or SQL
Application / Service	Idle-cost use case and dashboard orchestration	Depend on a specific broker/database where avoidable
Domain	Telemetry schemas and deterministic cost rules	Import framework or infrastructure clients
Infrastructure	SQL/NoSQL/cache repositories, Kafka consumer/producer	Leak storage-specific behavior into API contracts

6.2 Design Principles Applied

- **SOLID:** *[Where applied (file / class) and what it gave you]*
- **12-Factor App:** *[Config in environment, stateless processes, disposability]*
- **Idempotency, Fail-Fast, Least Privilege, DRY, KISS:** *[Where applied]*

6.3 Design Patterns Used

Pattern	Problem It Solves in Your System	Location in Code
Repository	Separates persistence from API/use-case code	backend/app/repositories/telemetry.py
Strategy / deterministic rule	Applies idle-cost logic from configured inputs	backend/app/services/idle_cost.py
Idempotent event key	Avoids duplicate event writes where keyed	backend/migrations/versions/936ae410a387_idempotent_intelligence_event_keys.py

6.4 Interfaces, Contracts & Runtime Flows

- **API Contract:** OpenAPI / AsyncAPI link; versioning, pagination, error format, rate limits

- **Event Schemas:** Topics, keys, schema format (Avro / Protobuf / JSON Schema), schema evolution
- **Sequence Diagrams:** Two critical flows, including one failure path (broker down, duplicate event, OEM format change)

API contract: FastAPI OpenAPI at /docs and /openapi.json; endpoints are versioned under /api/v1. Authentication uses bearer tokens; APIs return JSON and standard HTTP error responses. Pagination/rate limiting are not implemented globally. Event contract: JSON validated by Pydantic, Kafka topic configured in infra; vehicle identifier is the partition key. Schema registry, Avro/Protobuf and formal compatibility gates are not implemented. Flow A: simulator publishes ? consumer validates ? stores raw/current state ? rule creates PostgreSQL alert/cost/recommendation ? authenticated dashboard reads. Flow B failure: malformed event is logged and committed so it does not block a valid event behind it on the same partition; this path was smoke-tested. Broker outage recovery is not verified.

6.5 Algorithms & Data Structures

- **Problem Solved:** e.g. nearest reachable charger, trip segmentation, VIN / DTC parsing
- **Algorithm Used:** Graph, dynamic programming, regex, streaming structures — justify the choice
- **Pseudocode:** For major functions
- **Time & Space Complexity:** Annotate all
- **Scale Tested:** Input size and measured runtime

Core algorithm: for each valid telemetry event, persist raw record, update the vehicle?s expiring latest-state key, then evaluate deterministic idle conditions; if qualified, derive estimated cost from reported fuel rate ? idle duration ? configured fuel price ? assumed avoidable fraction and persist linked alert/action records. Pseudocode: validate(event); raw_store(event); redis_set(vehicle_id,event,TTL); if idle_rule(event): cost = rate ? duration ? price ? avoidable_fraction; upsert_alert_cost_action(event,cost). Per event, rule evaluation is O(1); storage is O(n) in retained raw events and O(v) hot keys for v active vehicles. Actual throughput is bounded by serial consumer/store operations and was not measured end-to-end. Scale evidence: 2,000 events per local producer rate step (100, 1K, 10K/sec targets); no production-scale runtime claim.

7. Non-Functional Requirements & Performance Benchmarks

NFR	Target (Case Study)	Achieved	How Measured
Ingest Throughput	100K+ events/sec case-study target	Not established; local producer smoke peaked at 9,218.3/sec at 10K target	2,000 event steps; producer/broker only
End-to-End Latency	<2 s dashboard; <5 s critical alert target	Not measured	No trace or event-to-screen timing
API Latency	p95 <200 ms; p99 <500 ms target	Not measured	No percentile collection
Resilience	Recover after broker/pod failure target	Partial: malformed message path handled; outage recovery not tested	Unit readiness fault simulation; no actual dependency outage
Availability	99.9%, no single point of failure target	Not applicable to single-node demo	No uptime measurement; single instance services

- **Load Test Setup:** Tool, data volume, hardware / cluster size
- **Results:** Graphs or screenshots of throughput, p95 / p99 and consumer lag

Environment: Windows local development host, Docker Compose single-node services (PostgreSQL, MongoDB, Redis, Kafka), 2,000 synthetic events per producer step; hardware CPU/RAM was not recorded. Measured producer rates: 100.0, 998.6 and 9,218.3 events/sec at the 100, 1,000 and 10,000 targets; Kafka delivery flush succeeded, and consumer lag was later sampled at zero across three partitions after catch-up. These measurements describe producer/broker smoke runs. API p95/p99, end-to-end event latency, per-step resource use, sustained 100K/s and five-minute 3? burst were not measured. See docs/PERFORMANCE.md and docs/FINAL_VERIFICATION.md.

8. Security & Compliance

- **Threat Model:** STRIDE for the ingestion path and public API; top five threats and controls
- **Authentication & Authorisation:** OAuth2 / OIDC, RBAC, tenant isolation
- **Device Identity & Encryption:** mTLS, TLS in transit, encryption at rest, secrets management
- **Privacy:** How location and driver data is minimised, masked, retained and erased (GDPR, DPDP Act 2023)
- **AI Safety (if applicable):** Prompt-injection defences, tool permissions, human approval, audit log

Threats and controls (STRIDE): Spoofed API identity ? signed bearer-token authentication; no OIDC integration. Tampered messages ? Pydantic schema validation and malformed-message logging. Repudiation ? application logs, but no immutable audit system. Information disclosure ? synthetic-only records and environment-based local secrets; TLS/encryption-at-rest and tenant isolation are not established for the demo. Denial of service ? no global rate limits or tested capacity controls. Elevation of privilege ? authenticated routes, but no fine-grained RBAC proof. Device identity/mTLS, GDPR/DPDP retention/erasure workflow, formal threat scan and compliance review are not implemented. Do not place real driver or location data in this demo. No AI tools/API run in the product.

9. Test Strategy

Test Type	Tools	No. of Tests	Coverage / Result	In CI?
Unit/API	pytest, TestClient	14 passed	Assertions pass; coverage % not collected	No CI pipeline
Integration & Contract	pytest + local Compose	2 integration tests passed	Service reachability and deduplication/Redis state	No CI pipeline
Acceptance (BDD)	Manual local browser walkthrough	5 main views	Login, Overview, Vehicles, Alerts, Actions, Analytics inspected	No CI pipeline
Performance / Load / Soak	Simulator producer	3 ? 2,000 events	100.0 / 998.6 / 9,218.3 events/sec; no p95/p99; 100K not run	No CI pipeline
Security (SAST, DAST, Dependency, Image)	Not run	0 scans	No scan report; do not infer clean findings	No
Compliance & Chaos	Unit readiness fault simulation	1 simulated dependency test	HTTP 503 when mocked dependencies unavailable; real chaos not run	No

- **Edge Cases Covered:** e.g. duplicate events, out-of-order data, unknown OEM format, broker outage
- **Sample Input / Output:** If helpful
- **Report Links:** Coverage, load-test and security scan reports

10. Observability

- **Dashboard Screenshot:** Throughput, consumer lag, error rate, latency percentiles
- **Troubleshooting Walk-Through:** How you would find the cause of a latency spike using logs, metrics and traces

[Write your answer here]

11. AI / ML Component (if used)

- **Purpose:** The decision the model or agent supports, and why rules alone were not enough
- **Data & Features:** Training or retrieval data, how it was generated or sourced, leakage checks
- **Model / Agent Design:** Model choice, vector store, tools the agent can call, how access is limited
- **Evaluation:** Metrics against a simple baseline
- **Guardrails & Cost:** Failure handling, hallucination checks, human-in-the-loop, cost and latency per request

No runtime AI/ML component is used. Idle detection and cost estimation are deterministic rules because they are transparent and require no training data. There is no model, vector store, agent, external LLM call or AI inference evaluation in the application.

12. Architecture Decisions, Risks & Future Enhancements

- **Architecture Decision Records:** 3–5 records: Context → Options Considered → Decision → Consequences (cover databases, messaging and one CAP trade-off)
- **Risks & Technical Debt:** Known limitations and shortcuts taken during the hackathon
- **Future Enhancements:** Next three steps from prototype to pilot

ADRs in docs/architecture/ record Kafka for replayable partitioned messaging, polyglot stores by access pattern, Redis for expiring hot state, deterministic cost rules and deferring vector/AI features. Risks: single-node Compose, no auth federation or granular tenant RBAC, no TLS/mTLS, no retention policy for Mongo raw events, synchronous store calls, no global API rate limit, no 100K end-to-end test, no measured p95/p99, no production backup/restore validation. Next steps: (1) instrument end-to-end latency, errors, resource use and consumer lag with dashboards; (2) add tenant-aware OIDC/RBAC, encryption and data retention/erasure; (3) benchmark a horizontally scaled broker/worker and pilot only with consented, governed fleet data.

13. Demo Video (5 Minutes Maximum)

Videos longer than 5:00 are cut at 5:00. Show the working product, not only slides. 1080p, clear audio, captions recommended; unlisted YouTube, Vimeo or Drive link.

Time	Segment	What to Show
0:00?0:30	Problem	State idle-cost visibility problem; label it as a product hypothesis
0:30?1:00	Solution	Explain WHY ? COST ? ACTION
1:00?3:00	Live Demo	Login, show synthetic vehicle state, alert, estimate and recommendation
3:00?4:15	Under the Hood	Show Kafka/store architecture and bounded load evidence; state limits
4:15?5:00	Impact & Next Steps	No measured savings claim; explain instrumentation and pilot steps

- Video Link: Not recorded. The five-minute segment table below is a recording outline; attach a real unlisted video URL before final portal submission.

14. Repository Checklist

- **README:** Problem, architecture image, quick start, environment variables, test commands, known issues
- **One-Command Run:** docker compose up (or make run) starts the full stack with the data simulator
- **Structure:** Clear service folders, plus /docs, /infra and /tests
- **CI Pipeline:** Build, lint, all test suites and security scans on every push
- **Hygiene:** No secrets committed, .env.example provided, commits from all members
- **Final Tag:** v1.0-submission

15. Conclusion

- Summary of key learnings
- Strengths of the solution
- Challenges tackled and solved

The project demonstrates a complete synthetic-data path from event ingestion to an explainable fleet action, with a usable authenticated dashboard and documented architecture. The strongest result is a locally verified vertical slice plus bounded producer/broker smoke evidence. Key challenges include keeping duplicate/malformed event handling safe and joining telemetry to cost/action explanations. Production readiness, real fleet impact and high-scale performance remain future validation work.

16. Declarations

- **Open-Source Components:** List with licences (an SBOM file in the repository is acceptable)
- **AI Tools Used:** Coding assistants or LLM APIs, and the parts of the work they were used for
- **Data:** Confirm all data is synthetic or public and contains no real personal data

Open-source components include FastAPI (MIT), React (MIT), PostgreSQL (PostgreSQL License), Apache Kafka (Apache-2.0), MongoDB Community Server (SSPL-1.0 for applicable post-2018 releases), Redis Community Edition (license depends on exact server version; current recent versions have RSALv2/SSPLv1 terms), SQLAlchemy (MIT), Pydantic (MIT), and Uvicorn (BSD-3-Clause). See requirements.txt/package-lock.json and upstream licenses; transitive dependencies and exact image tags should be checked before redistribution. AI tools: OpenAI Codex assisted implementation/documentation; no LLM is called at runtime. Data: all included demo/simulator records are synthetic; no real personal data or public fleet dataset is included.

17. Appendix (if any)

- Additional diagrams or flowcharts
- Full test logs or large datasets used
- References or external sources